HS3 Algorithm Technical Documentation

X/Y obstacle transformation, third-order Hermite-Simpson transcription, solver flow, validation, and complete MATLAB file reference

Repository branch	`HS3-planner`
Documented commit	`8dbdeca`
Snapshot date	2026-08-27
Scope	93 tracked MATLAB files and 17 tracked support artifacts
Coordinate convention	Euclidean [x, y] in coordinate units

Snapshot rule. This document describes the tracked repository at the commit above. The untracked `RogueExamples/` directory is intentionally excluded because it is not part of that commit. No planner behavior was changed while producing this documentation.

Terminology edition: 2026-09-07. Coordinate names and units follow the generic x/y API. The algorithm discussion remains historical; consult the repository README for the current planner and independent wrapping options. Both archived PDF filenames are regenerated from this common source.

This is implementation documentation, not a claim of global completeness or global optimality. The bounded topology portfolio and local nonlinear solve limitations are stated explicitly in Section 9.

Contents

1 Purpose, scope, and reading map

This report explains how the repository turns time-indexed protected polygons in an x/y plane into a dynamically feasible, independently validated trajectory. It covers two related products:

- **Obstacle-aware X/Y planner**: the public `obstacleAvoidance.planTrajectory` API, its geometry/search layer, its planner-specific corridor constraints, and its independent collision validator.
- **Dimension-neutral HS3 engine**: the public `solveTrajHS3` boundary-value solver and the shared `hs3Internal` polynomial, constraint, and numerical solver components.

The obstacle-aware planner deliberately does not call the public `solveTrajHS3` wrapper. It calls the neutral `hs3Internal` primitives through its own adapter because moving/deforming obstacle associations, route topology, relinearization, and adaptive polygon collision validation belong to the X/Y layer. This is an architectural boundary, not a duplicate implementation.

The recommended reading order is: architecture (Section 2), mathematics (Section 3), obstacle transformation (Section 5), the worked example (Section 7), and finally the exhaustive file catalog (Section 10).

1.1 Evidence convention

Source references name a tracked file and line range from commit `ce0dfdd42c5cf186761603fa18d14c5ba4583551`. Mathematical equations are restatements of implemented formulas. Illustrative numeric values in the walkthrough are explicitly marked as hand-calculated examples; they are not presented as measured optimizer output. The repository commit is the primary authority for implementation-specific claims in this report[1]; external publications are cited where they explain the underlying method, but they do not establish what this particular implementation does.

2 System architecture

2.1 Two-product ownership model

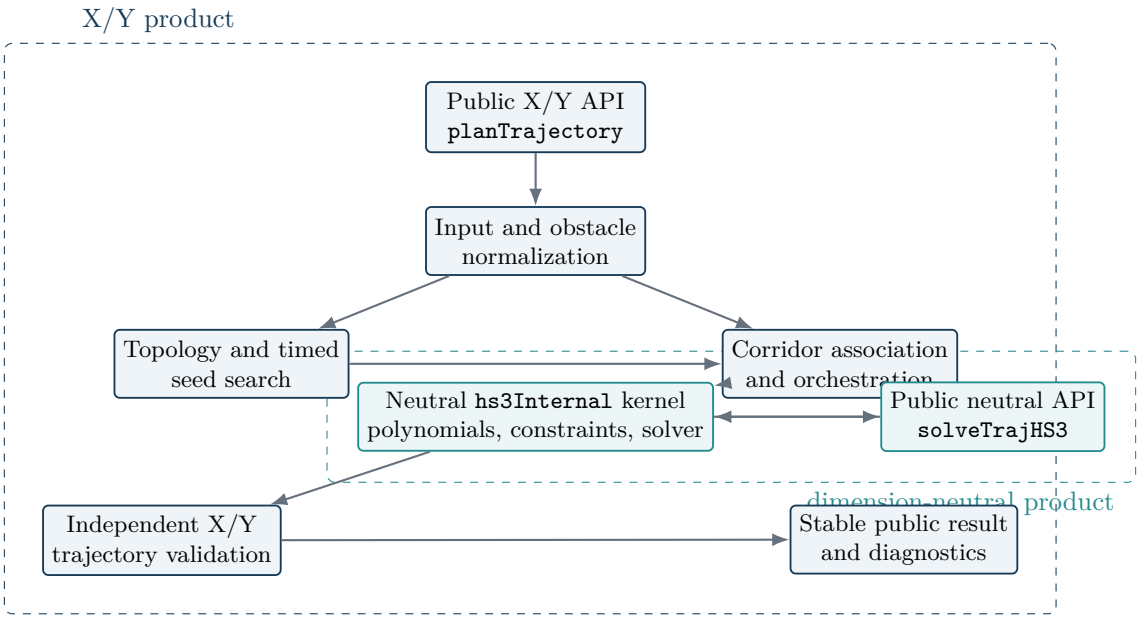

Figure 1: Ownership and dependency direction. Polygon semantics never enter the neutral HS3 source. The public neutral wrapper and the X/Y planner share the same kernel but have different orchestration and validation responsibilities.

2.2 Public planner contract

The maintained public call is:

```
result = obstacleAvoidance.planTrajectory( ...
    obstacles, initialState, goalState, limits, options);
```

The input roles are fixed and ordered:

- `obstacles`: canonical static or time-indexed protected polygon records, a nested cell/array accepted by the combiner, or empty.
- `initialState`: absolute `time_s` and a 1-by-2 `position_units`; optional endpoint velocity and acceleration default to zero.
- `goalState`: fixed or sampled moving target with the same coordinate ordering and supported terminal derivatives.
- `limits`: per-axis velocity, acceleration, and jerk limits plus optional workspace intervals.
- `options`: algorithm and runtime controls. Partial structures are accepted; defaults have one owner.

Expected planning outcomes do not throw. They return one stable schema with `Success=false`, an actionable `Message`, a machine-readable `TerminationReason`, resolved inputs/options, search evidence, candidate summaries, validation, and timing. Invalid input contracts throw identified errors.

2.3 End-to-end planner flow

3 HS3 mathematical formulation

3.1 What HS3 means in this repository

The repository explicitly calls the method a **third-order Hermite-Simpson (HS3) transcription**. Its separated higher-order state-chain interpretation is consistent with the collocation formulation discussed by Moreno-Martin, Ros, and Celaya[2]. For D coordinates and N equal-time segments, the implemented decision is a shared chain of jerk ordinates

$$J = \{j_0, j_{1/2}, j_1, j_{3/2}, \ldots, j_N\} \in \mathbb{R}^{(2N+1)\times D}. \tag{1}$$

For earliest-arrival problems, the absolute final time t_f is appended to the decision vector. For fixed arrival, t_f is data and the jerk controls are the only decisions. Adjacent segments share their endpoint jerk ordinate.

On segment k, let $\sigma \in [0, 1]$ be local normalized time and let $h = (t_f - t_0)/N$. If j_s, j_m, and j_e are the segment start, midpoint, and end jerk ordinates, the implemented quadratic jerk is

$$j_k(\sigma) = j_s + (-3j_s + 4j_m - j_e)\sigma + (2j_s - 4j_m + 2j_e)\sigma^2. \tag{2}$$

This is exactly the formula in `hs3/+hs3Internal/+polynomial/createTrajectoryPolynomial.m`, lines 31-35. Exact integration with the inherited state at the beginning of the segment produces cubic acceleration, quartic velocity, and quintic position. State is propagated from one segment to the next, so position, velocity, and acceleration remain continuous.

3.2 Fixed-time objective and affine structure

For fixed duration, position, velocity, acceleration, jerk, and terminal state are affine functions of the jerk vector. The repository builds those maps directly in `createAffineSensitivityModel.m`. The exact integrated squared-jerk objective for one coordinate and one segment is

$$I_k = \frac{h}{30}\left(4j_s^2 + 16j_m^2 + 4j_e^2 + 4j_s j_m - 2j_s j_e + 4j_m j_e\right). \tag{3}$$

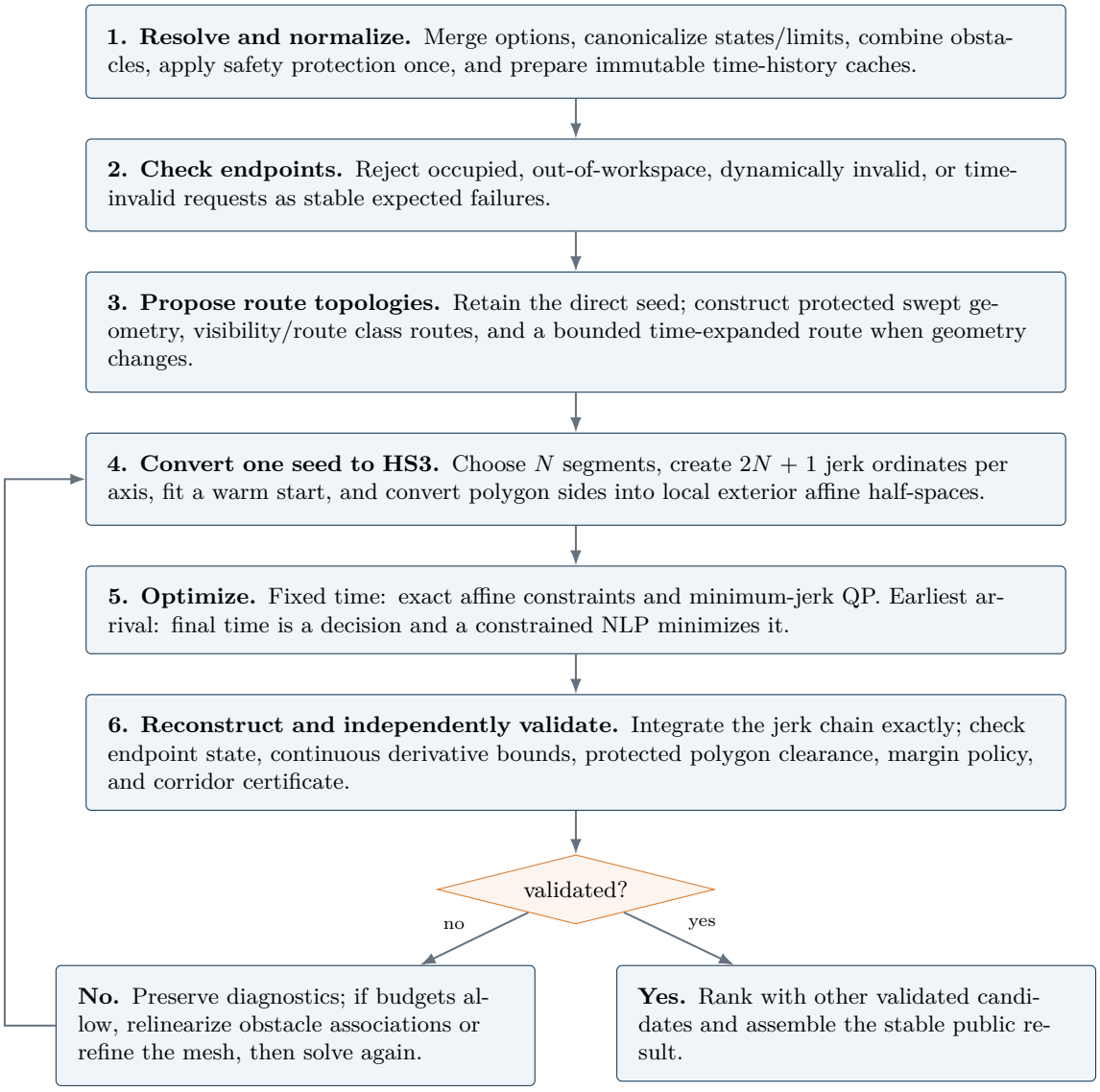

Figure 2: The maintained obstacle-aware planning sequence. Search proposals and optimizer feasibility are not accepted as proof; independent validation is the authority.

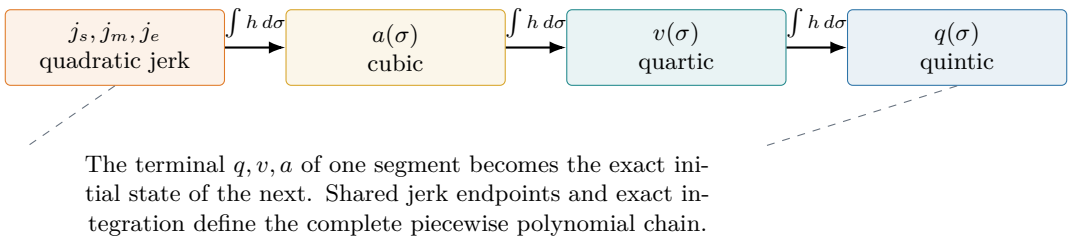

Figure 3: Polynomial orders created by the HS3 control-jerk representation.

Summing Equation (3) over segments and axes produces a positive semidefinite quadratic objective with an exact gradient and Hessian. Endpoint constraints and all fixed-time Bernstein/path constraints are affine in J. The neutral fixed-time engine therefore solves a convex quadratic program with `quadprog`[6]. The obstacle-aware adapter uses the same mathematical structure once its local half-space associations are frozen.

3.3 Earliest-arrival objective

For earliest arrival, t_f changes h, which changes every integrated coefficient map, obstacle event coordinate, and continuous bound. The objective is simply

$$\min_{J,t_f} t_f. \tag{4}$$

Jerk derivatives of the constraints remain exact. The final-time derivative is computed by a safeguarded finite difference that stays within allowed time and obstacle-event bounds. The local nonlinear problem is solved with `fmincon`[7]. The engine first obtains a fixed-horizon seed; if the primary solve is not feasible it may run a bounded feasibility-recovery stage.

3.4 Continuous bounds through Bernstein coefficients

Every segment polynomial is stored in ascending power form. For continuous position, velocity, acceleration, jerk, and affine corridor checks, the code converts the relevant polynomial to Bernstein form on the full segment or an owned subinterval. The complete-interval argument uses the Bernstein convex-hull property[3]. If a scalar polynomial is written

$$f(\sigma) = \sum_{r=0}^{d} \beta_r B_{r,d}(\sigma), \tag{5}$$

then the Bernstein convex-hull property gives

$$\min_r \beta_r \le f(\sigma) \le \max_r \beta_r, \qquad \sigma \in [0,1]. \tag{6}$$

Thus checking every β_r against an upper/lower limit certifies the complete polynomial interval, not only a sampling grid. Subinterval maps are exact and are restricted to one HS3 segment.

3.5 Power-to-Bernstein conversion in detail

The conversion is a change of basis; it does not approximate or resample the polynomial. Let the ascending-power record for one segment be

$$p(\sigma) = \sum_{k=0}^{d} a_k \sigma^k, \qquad 0 \le \sigma \le 1. \tag{7}$$

Using

$$\sigma^k = \frac{1}{\binom{d}{k}} \sum_{i=k}^{d} \binom{i}{k} B_{i,d}(\sigma), \tag{8}$$

the same polynomial has Bernstein coefficients

$$\boxed{\beta_i = \sum_{k=0}^{i} \frac{\binom{i}{k}}{\binom{d}{k}} a_k}, \qquad i = 0, \ldots, d. \tag{9}$$

Equation (9) is exactly the lower-triangular conversion assembled by the following implementation owner:

`hs3/+hs3Internal/+polynomial/convertPowerToBernstein.m`, lines 26–38.

MATLAB's lower-triangular Pascal matrix supplies $\binom{i}{k}$ and columnwise division by its last row supplies $\binom{d}{k}^{-1}$. The matrix is cached by degree because position, velocity, acceleration, jerk, sensitivities, and corridor projections repeatedly use the same basis change.

For a concrete quintic example,

$$p(\sigma) = 1 + 2\sigma - 3\sigma^2 + 0.5\sigma^5, \quad a = [1, 2, -3, 0, 0, 0.5]^T. \tag{10}$$

Applying Equation (9) gives

$$\beta = [1, 1.4, 1.5, 1.3, 0.8, 0.5]^T. \tag{11}$$

The negative power coefficient $a_2 = -3$ does not mean the curve reaches -3; raw power coefficients are not geometric control values. In contrast, the Bernstein coefficients prove $0.5 \le p(\sigma) \le 1.5$ over the entire interval. The bound can be conservative - the curve need not attain either extreme - but it is continuous and cannot miss a between-sample peak.

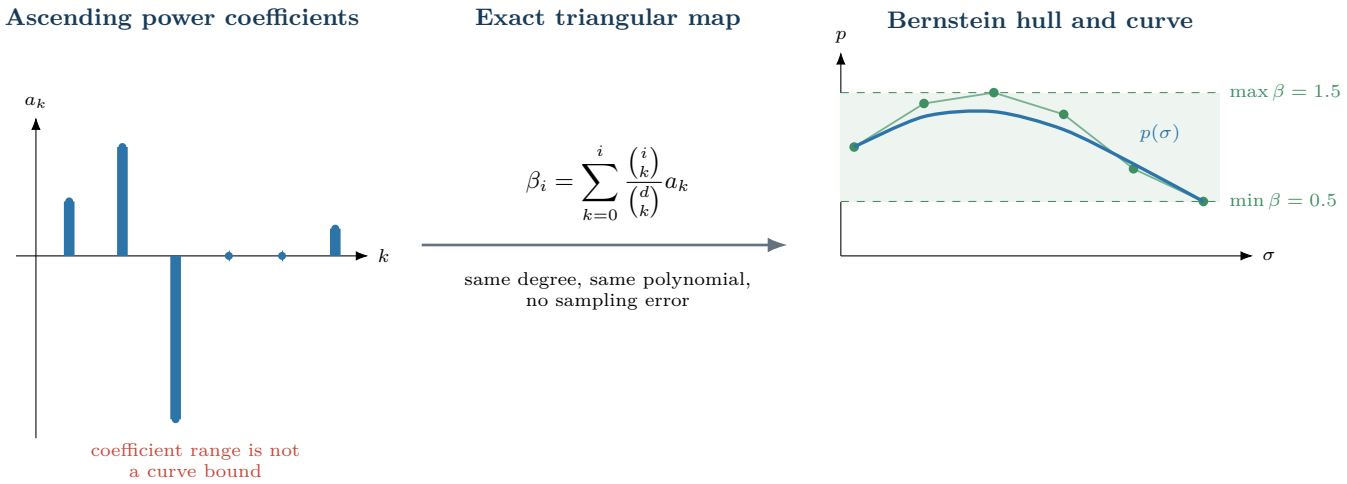

Figure 4: Exact power-to-Bernstein conversion for the worked quintic. The raw power coefficients on the left are algebraic coefficients; the Bernstein coefficients on the right are geometric control values whose convex hull contains the complete curve.

For a constraint that owns only $\tau \in [\tau_a, \tau_b]$ inside one HS3 segment, `createSubintervalBernsteinMap.m` first substitutes $\sigma = \sigma_a + (\sigma_b - \sigma_a)u$, $u \in [0, 1]$, to obtain an exact power record on that subinterval, then applies the same conversion. A point row ($\tau_a = \tau_b$) needs one evaluated value; a nonzero interval emits all $d + 1$ Bernstein inequalities.

Intervals are prohibited from crossing an HS3 segment boundary so one polynomial record always owns the complete proof.

In the fixed-time problem, the power coefficients are affine in jerk. The code therefore applies the same triangular basis map to the coefficient *sensitivity matrices*, not just to one trial curve. This yields exact linear inequality rows for `quadprog`; the solver does not numerically differentiate those jerk columns.

3.6 Endpoint and path constraints

The terminal equality is

$$q(t_f) = q_f, \qquad v(t_f) = v_f, \qquad a(t_f) = a_f. \tag{12}$$

The neutral public path-constraint row has fields `Tau`, `TauEnd`, `Normal`, and `LowerBound`. A point row enforces

$$n^T q(\tau) \geq \ell, \tag{13}$$

while TauEnd > Tau applies the same half-space throughout the interval using its Bernstein hull. The X/Y planner constructs the equivalent rows internally from obstacle geometry.

4 Create 1D, 2D, and 3D HS3 inputs

The neutral `solveTrajHS3` function uses the width of the state vectors to find the number of coordinates. Use a row with D values for every state and limit that changes by coordinate. A 1D input has one value, a 2D input has two values, and a 3D input has three values. Do not mix widths in one request.

Input	Required contents
`initialState`	Scalar structure with `time`, `position`, `velocity`, and `acceleration`. The three motion fields are $1 \times D$ rows.
`terminalState`	Scalar structure with `position`, `velocity`, `acceleration`, and `maximumTime`. The motion fields are $1 \times D$ rows.
`limits`	Positive `maximumVelocity`, `maximumAcceleration`, and `maximumJerk`. Each value can be one scalar for all coordinates or a $1 \times D$ row.
`options`	Use `TimeMode="fixed"` and set `FinalTime` for a fixed-duration example. Use `TimeMode="free"` to let HS3 find an early arrival before `terminalState.maximumTime`.

All examples below start and finish at rest. They use generous limits so that the input layout is easy to see. Each example checks `trajectory.Success` and prints the machine-readable termination reason when the solve fails.

4.1 1D example: move one coordinate

Use one value in each motion row. This example moves from 0 to 1 in four seconds.

```
addpath("hs3");

initialState = struct( ...
    "time", 0, ...
    "position", 0, ...
    "velocity", 0, ...
    "acceleration", 0);

terminalState = struct( ...
    "position", 1, ...
    "velocity", 0, ...
    "acceleration", 0, ...
    "maximumTime", 4);
```

```
limits = struct( ...
    "maximumVelocity", 2, ...
    "maximumAcceleration", 3, ...
    "maximumJerk", 8);

options = struct("TimeMode", "fixed", "FinalTime", 4);
trajectory = solveTrajHS3( ...
    initialState, terminalState, limits, options);

assert(trajectory.Success, trajectory.Message);
plot(trajectory.time, trajectory.position(:, 1));
xlabel("Time");
ylabel("Coordinate 1");
```

The returned histories have one column. For example, `trajectory.position(:,1)` is the complete position history for the only coordinate.

4.2 2D example: move two coordinates

Use two values in each motion row. The first column is coordinate 1. The second column is coordinate 2. This example moves to $[1, 2]$ in five seconds.

```
addpath("hs3");

initialState = struct( ...
    "time", 0, ...
    "position", [0 0], ...
    "velocity", [0 0], ...
    "acceleration", [0 0]);

terminalState = struct( ...
    "position", [1 2], ...
    "velocity", [0 0], ...
    "acceleration", [0 0], ...
    "maximumTime", 5);

limits = struct( ...
    "maximumVelocity", [2 2], ...
    "maximumAcceleration", [3 3], ...
    "maximumJerk", [8 8]);

options = struct("TimeMode", "fixed", "FinalTime", 5);
trajectory = solveTrajHS3( ...
    initialState, terminalState, limits, options);

assert(trajectory.Success, trajectory.Message);
plot(trajectory.position(:, 1), trajectory.position(:, 2));
xlabel("Coordinate 1");
ylabel("Coordinate 2");
axis equal;
```

For x/y work, coordinate 1 can be x and coordinate 2 can be y. The neutral HS3 function does not assign those meanings. The caller must use consistent units and coordinate order.

4.3 3D example: move three coordinates

Use three values in each motion row. This example moves to $[1, -1, 0.5]$ in six seconds.

```
addpath("hs3");

initialState = struct( ...
    "time", 0, ...
    "position", [0 0 0], ...
    "velocity", [0 0 0], ...
    "acceleration", [0 0 0]);

```

```
terminalState = struct( ...
    "position", [1 -1 0.5], ...
    "velocity", [0 0 0], ...
    "acceleration", [0 0 0], ...
    "maximumTime", 6);

limits = struct( ...
    "maximumVelocity", [2 2 2], ...
    "maximumAcceleration", [3 3 3], ...
    "maximumJerk", [8 8 8]);

options = struct("TimeMode", "fixed", "FinalTime", 6);
trajectory = solveTrajHS3( ...
    initialState, terminalState, limits, options);

assert(trajectory.Success, trajectory.Message);
plot3(trajectory.position(:, 1), ...
    trajectory.position(:, 2), ...
    trajectory.position(:, 3));
xlabel("Coordinate 1");
ylabel("Coordinate 2");
zlabel("Coordinate 3");
axis equal;
```

The returned position, velocity, acceleration, and jerk histories have three columns. The row count is the number of returned time samples.

4.4 Change limits by coordinate

A scalar limit applies to all coordinates. A row gives each coordinate its own limit. For example, the following 3D limits allow coordinate 1 to move faster than coordinates 2 and 3:

```
limits.maximumVelocity = [4 2 1];
limits.maximumAcceleration = [6 3 2];
limits.maximumJerk = [12 8 4];
```

If input validation reports a size error, compare the width of every position, velocity, acceleration, limit, and path-normal row. They must all describe the same value of D. If the sizes are correct but no motion is found, inspect `trajectory.TerminationReason`, `trajectory.Message`, and `trajectory.MaximumConstraintViolation`. Then increase the final time or correct limits only when the physical request permits that change.

5 From an X/Y obstacle to HS3 input

5.1 Canonical obstacle representation

Obstacle coordinates are treated as Euclidean $[x, y]$ coordinates in coordinate units. Distances and path lengths are therefore planar degree distances, not spherical or geodesic distances.

The constructor retains two histories:

- `originalX_units`/`originalY_units`: source geometry;
- `x_units`/`y_units`: protected geometry used by planning and collision checking.

A requested safety margin m is applied once, absolutely, from the original geometry with a square-joint polygon buffer. Rebuilding with a different margin starts from the original boundary; it does not buffer an already buffered polygon. Exact rigid translations may reuse a translated copy of the first protected slice. The underlying MATLAB buffer operation is documented by MathWorks[8]; the margin-once ownership rule is specific to `+obstacleAvoidance/+obstacles/createObstacle.m`, lines 393-565.

5.2 Time interpolation and topology changes

After construction, `prepareObstacles.m` caches shapes, bounds, interval data, and vertex speed bounds. When adjacent slices have equal vertex count and matching finite/NaN masks, each vertex is interpolated linearly:

$$V_i(t) = V_i(t_k) + \lambda\,[V_i(t_{k+1}) - V_i(t_k)], \qquad \lambda = \frac{t - t_k}{t_{k+1} - t_k}. \tag{14}$$

When topology differs, the code does not invent vertex correspondence. It uses the conservative union of the two endpoint shapes throughout that interval. A single-slice obstacle is active for every query time; a multi-slice obstacle is inactive outside its stored time span.

5.3 Transformation pipeline graphic

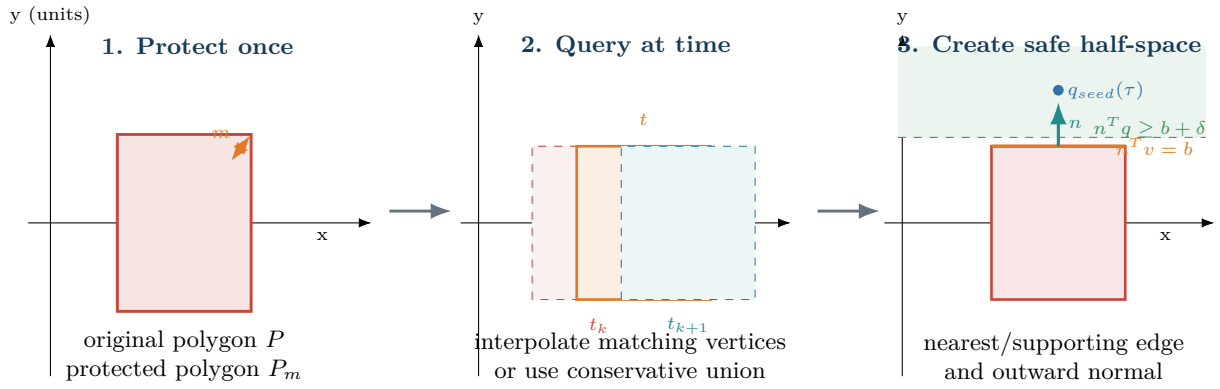

Figure 5: Obstacle-to-HS3 geometry transformation. Protection is already in the polygon before HS3. At an association time, the seed chooses an exterior side and the protected polygon becomes a local affine half-space.

5.4 Seed topology before optimization

The direct start-to-goal seed is always retained as a proposal. For obstacles, the search layer samples source times, interval midpoints, and bounded uniform times, then unions protected shapes into a swept planar shape. Boundary vertices, start, and goal become visibility nodes. Candidate vertices are offset outward by at least 10^{-3} degree and clipped to the workspace. Delaunay edges provide a sparse candidate set with a bounded exhaustive fallback; visible edges use Euclidean degree length.

The graph state is augmented with a bounded two-dimensional winding/route class signature so distinct sampled route classes can be retained. This is an implementation-specific planar adaptation inspired by search with homotopy class constraints[4]; it is not the cited paper's full three-dimensional construction and is not a continuous space-time homotopy certificate.

For changing geometry, a forward time-expanded graph uses state (time layer, visibility node). Wait and motion edges move to a later layer. A 13-point check filters each motion proposal, but that check is only a search heuristic; it is never the final collision certificate.

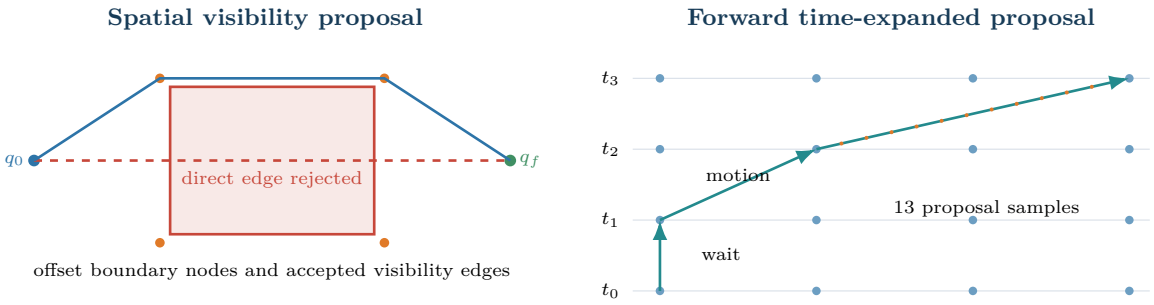

Figure 6: The spatial and timed graph stages propose topology and event timing. They do not certify the final polynomial trajectory.

5.5 Polygon side to affine HS3 row

At dense association coordinates τ, including obstacle event times, the planner interpolates the seed and queries each protected obstacle. For selected edge $e = B - A$, the unit left normal is

$$\ell = \frac{[-e_y, e_x]}{\|e\|_2}. \tag{15}$$

The stored outward sign s (or a probe-based sign for complex geometry) gives $n = s\ell$. For a convex supporting plane,

$$b = \max_{v \in P_m(t)} n^T v. \tag{16}$$

For a selected non-supporting edge, $b = n^T A$. The required separation is $\delta = \max(10^{-4} \text{ deg}, \epsilon_{collision})$, and the optimizer inequality is

$$c(\tau) = b + \delta - n^T q(\tau) \le 0, \quad \text{equivalently} \quad n^T q(\tau) \ge b + \delta. \tag{17}$$

Stationary associations may own a complete subinterval. Projecting the HS3 position polynomial onto n and converting it to Bernstein coefficients gives

$$c_r = b + \delta - \beta_r(n^T q) \le 0 \quad \text{for every coefficient } r. \tag{18}$$

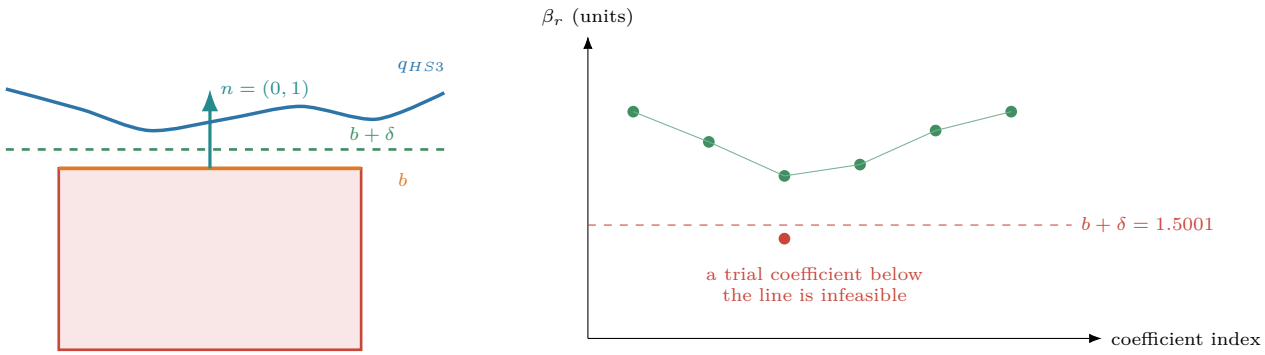

Figure 7: A stationary obstacle support becomes a complete-interval Bernstein constraint. Every green coefficient above the threshold certifies the entire projected polynomial span.

5.6 Moving obstacles and the authority of validation

Moving/deforming geometry uses ordered frozen-time associations during the local solve. A fast-moving association can be locked while the seed continues to satisfy it. These rows guide the optimizer but are not treated as a complete continuous collision proof.

Independent validation splits every HS3 segment at obstacle source times, checks endpoints, and adaptively bisects intervals. If midpoint clearance is d_m, the conservative relative-motion allowance for interval width Δt is

$$r = (\|v_{path,max}\|_2 + v_{obs,max})\,\frac{\Delta t}{2}. \tag{19}$$

The interval is certified only when $d_m > r + \epsilon_{collision}$. Otherwise it is bisected. If the minimum allowed time step is reached without proof, validation fails; it never silently accepts an unresolved interval.

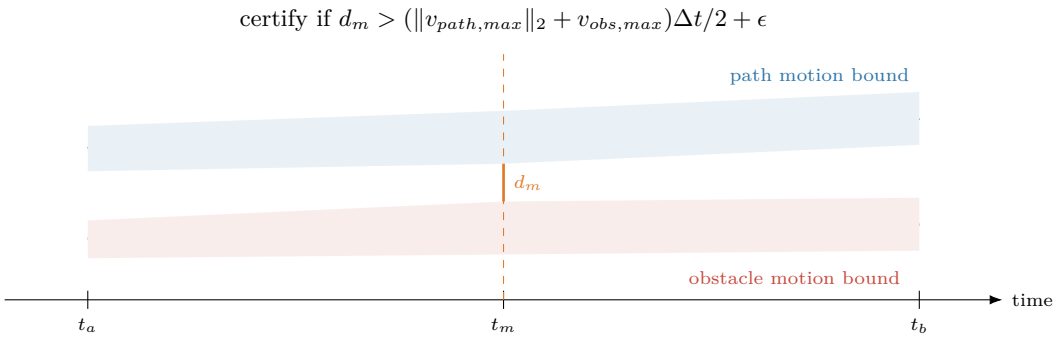

Figure 8: Adaptive collision certificate. Failure to establish separation causes bisection; an unresolved minimum-size interval is a validation failure.

5.7 Coordinate wrapping limitation

The current coordinate interface has independent `WrapX` and `WrapY` flags. Obstacle-free fixed-position requests select the nearest equivalent goal on each enabled axis using that axis's workspace interval width as its period. Half-period ties select positive displacement. Returned motion remains continuous; spatial plots split at either periodic seam. Obstacles and moving goals remain unsupported when either flag is enabled. Graphics must therefore not be read as a periodic obstacle transformation.

6 Solver, candidate, and result logic

6.1 Warm start from a topology seed

A route seed is parameterized by cumulative planar arc length to obtain $q_{seed}(\tau)$. The solver samples it at the HS3 control coordinates and fits jerk ordinates by regularized least squares using exact affine sensitivity maps. Desired velocity follows local route tangents, is bounded by axis limits, and respects endpoint derivatives. The fitted jerk is clipped to the configured limit and used only as an initialization; it does not define reachability.

6.2 Fixed and free solve paths

Property	Fixed arrival	Earliest arrival
Decision	J	J and t_f
Objective	Integrated squared jerk, Equation (3)	Absolute final time
Constraint structure	Affine in jerk after corridor association	Nonlinear because duration changes maps and geometry time
Primary numerical method	Convex QP through `quadprog`	Local NLP through `fmincon`
Derivative policy	Exact matrices and exact objective gradient/Hessian	Exact jerk columns; safeguarded final-time difference
Candidate ranking	Shortest validated sampled motion, then jerk	Earliest within arrival tolerance, then jerk

6.3 Mesh refinement and corridor relinearization

The planner can retry a candidate with updated obstacle associations and can increase the segment count within configured work limits. These are bounded attempts. They do not imply convergence, completeness, or discovery of every homotopy class. Diagnostics retain every attempted seed, mesh, relinearization, solver state, validation state, and relevant rejection count.

6.4 Independent acceptance rule

An optimizer-feasible result is only a candidate. Public acceptance requires the independent validator to pass:

- stable finite histories and a strictly increasing time base;
- exact initial and terminal position/velocity/acceleration agreement;
- workspace and full continuous position/velocity/acceleration/jerk bounds from the returned polynomial;
- continuous collision clearance against protected geometry across obstacle time history;
- safety-margin and coordinate-wrap policy consistency;
- a valid static seed-corridor certificate where applicable.

The returned `Success` becomes true only after this validation. Constraint violations are reported; the trajectory is not clipped to hide them.

7 Simple rectangle walkthrough

This section uses small hand-calculated values to show the data transformation. It is an explanatory trace, not recorded optimizer output.

7.1 Step 1: define the physical request

Let the original static obstacle be

$$P = [4, 6] \times [-1, 1] \text{ deg}, \qquad t = [0] \text{ s}, \tag{20}$$

with safety margin $m = 0.5$ degree. Let

$$\begin{aligned} q_0 &= (0, 0) \text{ deg}, & t_0 &= 0 \text{ s}, \\ q_f &= (10, 0) \text{ deg}, & t_f &= 12 \text{ s}, \end{aligned}$$

and use zero endpoint velocity/acceleration, velocity limit $(2, 2)$ units/s, acceleration limit $(1, 1)$ units/s^2, and jerk limit $(2, 2)$ units/s^3.

7.2 Step 2: apply the margin once

The square-joint buffer produces the protected rectangle

$$P_m = [3.5, 6.5] \times [-1.5, 1.5] \text{ deg}. \tag{21}$$

Both the original and protected coordinates remain in the obstacle record. Every downstream search, corridor, occupancy, and collision operation uses P_m; it does not add another 0.5 degree.

7.3 Step 3: detect that the direct seed is blocked

The direct spatial seed is $q(\tau) = (10\tau, 0)$. It intersects P_m for $\tau \in [0.35, 0.65]$. The direct seed remains in diagnostics as a proposal, but it cannot pass collision validation.

7.4 Step 4: create a detour topology

The protected swept shape equals P_m because the obstacle is static. A visibility route can pass above or below. For arithmetic, take the conceptual top seed

$$(0, 0) \to (3.5, 2.0) \to (6.5, 2.0) \to (10, 0). \tag{22}$$

The actual graph offsets boundary vertices by at least 0.001 degree; exact vertex order depends on MATLAB's polygon boundary traversal. The route is parameterized by cumulative arc length, producing $q_{seed}(\tau)$.

7.5 Step 5: create HS3 decisions

With the default $N = 10$ segments, each axis has $2N + 1 = 21$ jerk ordinates. The fixed-time decision therefore has 42 values. Earliest arrival would append one final-time value. The warm-start fit uses the seed at control coordinates, but the optimizer is free to move the trajectory while satisfying its constraints.

7.6 Step 6: turn the top obstacle edge into a row

At a seed point near $(5, 2)$, the nearest protected edge is the top edge $y = 1.5$. Its outward normal and support are

$$n = (0, 1), \qquad b = 1.5 \text{ deg}. \tag{23}$$

With $\delta = 10^{-4}$ degree, Equation (17) becomes

$$[0 \quad 1]q(\tau) \geq 1.5001, \quad \text{or} \quad 1.5001 - \mathrm{y}(\tau) \leq 0. \tag{24}$$

Suppose the projected quintic position on the associated segment has Bernstein coefficients

$$[2.00, 1.80, 1.65, 1.70, 1.90, 2.00] \text{ deg}. \tag{25}$$

Every coefficient exceeds 1.5001 degree, so the complete segment is certified above that support. A trial coefficient of 1.49 degree would violate the row by 0.0101 degree and be infeasible.

7.7 Step 7: optimize and reconstruct

For the fixed 12-second request, the solver minimizes integrated squared jerk subject to endpoint state, continuous kinematic, topology corridor, and obstacle support constraints. Equation (2) is integrated exactly to reconstruct position, velocity, acceleration, and jerk histories. Sampled output contains uniform points plus every polynomial knot.

7.8 Step 8: independently validate and select

The public validator rechecks the returned polynomial, derivative bounds, and protected polygon clearance. A top and bottom route may both validate. Fixed arrival selects the shortest validated sampled motion, breaking ties with integrated squared jerk. If none validates, the planner returns `Success=false`, `TerminationReason="noValidatedSeed"`, and preserves the best partial candidate and search diagnostics.

7.9 Moving extension

If the same original rectangle translates 2 coordinate units in x between $t = 0$ and $t = 10$ seconds, the protected vertices translate identically. At $t = 5$ seconds, $\lambda = 0.5$ and

$$P_m(5) = [4.5, 7.5] \times [-1.5, 1.5], \qquad v_{obs,max} = 0.2 \text{ units/s}. \tag{26}$$

For a validation interval $\Delta t = 0.1$ second and path speed bound $\|v_{path,max}\|_2 = 2$ units/s, Equation (19) gives $r = (2 + 0.2)(0.05) = 0.11$ degree. Midpoint clearance must exceed $0.11 + \epsilon_{collision}$ or the interval is bisected.

8 Defaults, diagnostics, and failure behavior

8.1 Obstacle-planner defaults at the documented commit

Option	Default	Meaning
`GoalTimeMode`	`earliestArrival`	Minimize arrival within the supplied horizon.
`SampleTime_s`	0.05	Returned uniform sampling interval; knots are also retained.
`WrapX`, `WrapY`	false	Independent periodic axes; periods are workspace interval widths.
`MaximumSeedCount`	5	Bounded topology proposal count.
`DirectOnly`	false	Permit visibility, route class, and timed seeds.
`ObstacleClusterDistance_units`	0	Disable nearby-region seed clustering.
`CollocationSegmentCount`	10	Initial equal-duration HS3 segment count.

Option	Default	Meaning
`MaximumCollocationSegmentCount`	64	Mesh-refinement cap.
`MaximumMeshRefinementPasses`	2	Bounded refinement attempts.
`MaximumNlpIterations`	300	Local solver iteration cap.
`MaximumFunctionEvaluations`	30000	Solver evaluation cap.
`MaximumPlanningTime_s`	115	Cooperative outer planning budget.
`DeterministicWorkBudget`	false	Wall-time budget remains active by default.
`ArrivalTimeTolerance_s`	10^{-3}	Arrival comparison/refinement tolerance.
`ConstraintTolerance`	10^{-7}	Numerical constraint tolerance.
`OptimalityTolerance`	10^{-7}	Local solver optimality tolerance.
`StepTolerance`	10^{-10}	Local solver step tolerance.
`CollisionClearanceTolerance_units`	10^{-7}	Clearance tolerance for validation.
`CollisionMinimumTimeStep_s`	0.00025	Fail-closed adaptive collision resolution floor.
`RandomSeed`	0	Echoed deterministic seed; search is deterministic.

The neutral `solveTrajHS3` defaults use generic names and a narrower scope: earliest arrival, $N = 10$, sample time 0.05, 300 iterations, 30,000 evaluations, 115 time units of solve budget, 10^{-3} arrival tolerance, 10^{-7} constraint/optimality tolerances, 10^{-10} step tolerance, and quiet output.

8.2 First-class diagnostics

Search diagnostics are collected while searching. They include nodes, accepted and rejected edges, time layers, parent/cost relationships, route class/timed coverage, work-limit evidence, seed provenance, and the best partial state. Candidate summaries retain optimizer status, maximum violation, collision resolution, clearance, mesh/relinearization counts, timing, and solver output. Seven exclusive stage-timing fields reconcile to an independently measured total.

8.3 Expected failures

Recognized expected outcomes include endpoint infeasibility, exact static no-route/no validated seed, time-window infeasibility, local optimizer infeasibility, solver time limit, arrival pinned to the horizon, and validation failure. The standalone engine additionally reports an infeasible time horizon. All use stable output schemas. Invalid dimensions, missing fields, malformed histories, unsupported wrapping, unknown planner method, or corrupt internal state use identified errors.

9 Known limitations and interpretation

- The deterministic topology portfolio is finite and is not complete. A returned no-path result means no configured proposal produced an independently valid motion; it is not a mathematical proof that no motion exists unless the exact static graph condition specifically applies.
- Earliest-arrival HS3 is a local nonlinear method. It does not prove global minimum arrival or global optimality.
- Static earliest-arrival planning may stop after the first independently validated seed to protect the time budget, so an unattempted topology may be faster.
- Arrival quality is mesh-limited. A coarse mesh may yield a later valid arrival; refinement costs more solver work.
- Moving/deforming obstacle rows are local frozen-time associations. Independent adaptive polygon validation is authoritative between those association times.

- Wrapping either axis with obstacles or moving goals is unsupported.
- One solver evaluation or callback may finish after the cooperative time limit, so measured elapsed time can overrun the nominal budget.
- Geographic and X/Y geometry is planar in coordinate units, not spherical.

10 Complete tracked MATLAB file catalog

This catalog covers all 93 tracked `.m` files at the documented commit. "Called by" descriptions identify principal ownership, not every test call.

10.1 Production: geometry

`+obstacleAvoidance/+geometry/boundaryToEdges.m`

Converts every connected `polyshape` boundary ring into deterministic N-by-2 start/end edge rows. It removes only an explicitly repeated closing vertex and then closes each valid ring. Clearance and visibility code use it to enumerate actual polygon segments.

`+obstacleAvoidance/+geometry/boundaryToShape.m`

Converts paired, NaN-separated x/y vectors to an unsimplified `polyshape`. It preserves ring correspondence and collinear vertices; fewer than three finite vertices yields empty geometry. Dynamic preparation, time queries, and corridor construction depend on this canonical conversion.

`+obstacleAvoidance/+geometry/canonicalBoundaryToEdges.m`

Turns a canonical geometry record into directed closed edges in source order. Moving obstacle association and constraint evaluation use the stable ordering to keep edge identity and outward direction reproducible.

`+obstacleAvoidance/+geometry/convexPolygonRegions.m`

Decomposes occupied polygon geometry into convex regions. Already-convex regions are kept; nonconvex components are triangulated. Seed-corridor and corridor-certificate logic require convex components so one supporting half-space has valid local meaning.

`+obstacleAvoidance/+geometry/pointPolygonClearance.m`

Computes signed Euclidean clearance, nearest boundary point, and deterministic edge index for N-by-2 query points. Positive is outside, negative is inside, and an empty shape gives infinity. Occupancy, clustering, corridors, validation, and tests share this implementation.

10.2 Production: input normalization

`+obstacleAvoidance/+input/goalPositionAtTime.m`

Evaluates either a fixed goal or an interpolated sampled moving goal at absolute times. Endpoint checks, route search, terminal constraints, intercept planning, validation, and plotting call it so target interpretation stays consistent.

`+obstacleAvoidance/+input/normalizeLogicalScalar.m`

Implements the repository-wide scalar logical or binary-numeric contract while preserving the owning caller's error identifier and field name. Options, plotting, obstacle, example, intercept, and sandbox controls reuse it.

`+obstacleAvoidance/+input/normalizePlannerRequest.m`

Canonicalizes the public obstacle/state/limit request. It combines obstacles, converts numeric orientations to 1-by-2 doubles, defaults endpoint derivatives, supplies workspace intervals, checks cross-field dimensions, and implements the limited obstacle-free x-wrap policy.

+obstacleAvoidance/+input/resolveHs3Options.m

Owns and validates all obstacle-aware HS3 defaults. Partial or empty overrides are merged through the shared resolver; unknown fields warn once and do not affect behavior. It also emits migration errors for fields whose ownership changed.

+obstacleAvoidance/+input/resolveOptions.m

A generic partial scalar-structure merge. It retains default field order, substitutes defaults for omitted/empty values, returns unknown names without applying them, and is used by public options owners and display/query helpers.

+obstacleAvoidance/+input/validatePlannerEndpoints.m

Performs the pre-search feasibility screen: start/goal occupancy, workspace bounds, derivative limits, time ordering, and goal-time policy. It returns an expected failure reason/message instead of throwing for a physically infeasible request.

10.3 Production: obstacle infrastructure

+obstacleAvoidance/+obstacles/combineObstacles.m

Flattens obstacle arrays, nested cells, and empty inputs into one canonical column array while preserving caller order. It delegates normalization to `createObstacle` and provides a schema-preserving empty array.

+obstacleAvoidance/+obstacles/createMovingObstacle.m

Evaluates a caller transform independently at every increasing source time and constructs one canonical moving/deforming obstacle plus history metrics. It assumes no rigid motion, shared vertex count, or shared topology.

+obstacleAvoidance/+obstacles/createObstacle.m

The sole public obstacle constructor, normalizer, and protection rebuilder. It validates NaN-separated rings, preserves original/protected histories, applies an absolute square-joint `polybuffer` margin once, reuses exact translated protection when possible, and returns the stable canonical schema.

+obstacleAvoidance/+obstacles/hasChangingHistory.m

Classifies whether obstacle geometry or activation span changes over the requested plan. Search uses this result to decide whether a time-expanded topology proposal is needed.

+obstacleAvoidance/+obstacles/prepareObstacles.m

Precomputes sample shapes, history bounds, interval unions, interpolation deltas, topology flags, and vertex speed bounds. Matching topology is interpolated vertexwise; topology-changing intervals use a conservative stationary union.

+obstacleAvoidance/+obstacles/queryObstacleOccupancyAtTime.m

The public point/time occupancy and signed-clearance query. It broadcasts compatible inputs, uses prepared broad-phase bounds, queries already-protected shapes, reports the first blocker and detailed provenance, and treats the boundary as occupied by default.

+obstacleAvoidance/+obstacles/shapeAtTime.m

Returns the active protected shape and metadata at one physical time. It reuses exact source slices, interpolates matching vertices, returns conservative unions for topology changes, and reports convexity, outward sign, and speed information used by corridor construction.

10.4 Production: planner

+obstacleAvoidance/+planner/createConstraintMatrices.m

Builds the exact fixed-duration HS3 inequality/equality Jacobians in solver row order. It combines continuous Bernstein bounds, terminal P/V/A equations, static seed corridors, frozen obstacle corridors, and optional direct-line collinearity equations through the affine sensitivity model.

+obstacleAvoidance/+planner/createEmptyResult.m

Creates the single stable obstacle-planner result, seed-summary, polynomial, validation, and search diagnostic templates. Every success and failure exit inherits identical field order and explicit empty values.

+obstacleAvoidance/+planner/createEmptySolverDiagnostics.m

Creates and finalizes the stable solver diagnostic schema, including distinct solver and outer elapsed times. Candidate construction uses it even when no numerical solve ran.

+obstacleAvoidance/+planner/evaluatePlannerPolynomial.m

Adapts the unit-bearing obstacle-planner polynomial record to the dimension-neutral polynomial evaluator and returns absolute time plus position, velocity, acceleration, and jerk histories.

+obstacleAvoidance/+planner/evaluateTrajectoryConstraints.m

Evaluates all planner inequalities/equalities and gradients. It adds continuous Bernstein kinematic bounds, seed/topology corridors, time-indexed obstacle associations, terminal state, exact jerk Jacobian columns, and a safeguarded variable-final-time column.

+obstacleAvoidance/+planner/plan.m

The maintained internal orchestrator. It resolves and normalizes, validates endpoints, prepares dynamic geometry, creates deterministic seeds, manages budgets, solves/relinearizes/ refines candidates, independently validates them, ranks valid outcomes, and assembles a stable success or evidence-rich failure.

+obstacleAvoidance/+planner/solveRouteCandidate.m

Transforms one route seed into an HS3 problem. It sets mesh/control layout, fits a bounded jerk warm start, creates topology and moving-obstacle corridor associations, invokes the neutral solver executor, reconstructs exact polynomials, samples the motion, and returns one stable candidate and solver diagnostics.

+obstacleAvoidance/+planner/stageTiming.m

Defines and reconciles the seven exclusive planner timing fields against an independently measured total. It rejects over-attribution and applies the finalized timing schema at the public boundary.

+obstacleAvoidance/+planner/validatePolynomialTrajectory.m

Checks the returned polynomial schema, time basis, continuity, endpoint state, sampled histories, and complete continuous coordinate/derivative bounds. It uses polynomial coefficients rather than trusting samples alone.

10.5 Production: search and certification

+obstacleAvoidance/+search/certifySeedCorridor.m

Independently verifies a solver-supplied static corridor certificate. It checks schema, complete obstacle-envelope containment, supporting half-spaces, and the entire returned polynomial through Bernstein inequalities; uncertainty fails closed.

+obstacleAvoidance/+search/createRouteCandidates.m

Creates the bounded deterministic route portfolio and first-class search diagnostics. It retains the direct proposal, builds swept geometry and visibility nodes/edges, adds a timed route for changing history, enumerates distinct route class classes, and only shortens a route when visibility and signature are preserved.

+obstacleAvoidance/+search/createSeedCorridor.m

Converts convex seed-envelope regions to per-segment exterior support records. A seed midpoint chooses the free side and a retained clearance; later certification verifies the complete polynomial rather than trusting that midpoint.

+obstacleAvoidance/+search/denseSweptEnvelope.m

Replaces an unaffordable exact swept union with a conservative directional-support hull for seed generation. It protects source endpoints, clips against support planes, and reports whether the approximation was used.

+obstacleAvoidance/+search/seedCorridorInequality.m

Projects each position polynomial onto its seed-corridor normal, converts to Bernstein form, and emits finite inequalities that enforce continuous exterior separation over the complete segment.

+obstacleAvoidance/+search/seedEnvelopeContainsObstacles.m

Checks that one connected corridor boundary contains a conservative continuous sweep of every protected obstacle. Incomplete containment prevents a static corridor from being treated as a certificate.

+obstacleAvoidance/+search/timeExpandedVisibilitySearch.m

Searches a forward layered graph over every supplied planning time; each state is a time layer plus visibility node. It supports waiting and motion arcs, uses sampled edge-collision filtering, and returns route positions, absolute route times, parents, costs, and work counts.

10.6 Production: public APIs and plotting

+obstacleAvoidance/planMovingTargetIntercept.m

Adapts a sampled target history to the maintained public trajectory planner. Specified-time mode runs one plan; earliest mode performs bounded chronological trials and refines only the first observed feasible bracket. Target derivatives are derived consistently from the sampled track.

+obstacleAvoidance/planTrajectory.m

The public HS3-only planner entry point and zero-input/defaults query. It validates the sole supported `PlannerMethod="hs3"`, removes the selector, and dispatches to the internal orchestrator under the stable public contract.

+obstacleAvoidance/validateTrajectory.m

The independent public validator. It combines polynomial/endpoint/continuous-bound checks with adaptive protected-polygon collision certification, safety-margin and wrap policy checks, and optional static corridor certification. Unresolved intervals fail rather than pass by sampling.

+obstacleAvoidance/+plotting/plotTrajectory.m

Creates spatial search/motion, animation, and position/velocity/acceleration/jerk views solely from the returned result. It distinguishes original/protected geometry and accepted/rejected search evidence, respects figure/animation controls, and never reruns planning.

10.7 Dimension-neutral HS3 engine

hs3/solveTrajHS3.m

The public D-dimensional fixed-time or earliest-arrival boundary-value solver. It owns the generic input contract, normalizes optional affine point/interval path rows, calls the fixed/free solvers, reconstructs and samples the polynomial, computes jerk cost, validates independently, and returns one stable schema.

hs3/+hs3Internal/defaultOptions.m

The single source of argument-independent neutral HS3 defaults for mode, mesh, sampling, iteration, evaluation, time, and numerical tolerances.

hs3/+hs3Internal/validate.m

Independently checks neutral result histories, strictly increasing time, initial/terminal P/V/A agreement, and complete continuous/path constraints by reevaluating the returned control decision.

hs3/+hs3Internal/+solver/optimize.m

Executes normalized numerical problems and produces a stable stage record. Fixed-time exact convex problems use `quadprog`; free-time problems use gradient-enabled `fmincon`; a bounded feasibility-recovery solve may follow. It centralizes exit flags, violations, time limit, and error capture.

hs3/+hs3Internal/+solver/solveFixedTime.m

Builds the fixed-duration minimum-integrated-squared-jerk QP: exact constraint values/matrices, exact jerk Hessian, bounds, and a minimum-norm equality-feasible initial decision. It then delegates numerical execution to `optimize`.

hs3/+hs3Internal/+solver/solveFreeTime.m

Builds the variable-final-time arrival NLP. It first obtains a deterministic fixed-horizon jerk seed, appends final time and its bounds, defines the exact arrival objective, and delegates the local solve/recovery to `optimize`.

hs3/+hs3Internal/+constraints/createFixedConstraintMatrices.m

Builds exact dimension-neutral jerk Jacobians for continuous coordinate and derivative bounds, terminal P/V/A, and optional affine point/subinterval path constraints. Each coordinate occupies its own control block.

hs3/+hs3Internal/+constraints/evaluateConstraints.m

Reconstructs the decision and returns inequalities $c \le 0$, equalities $c_{eq} = 0$, and `fmincon`-oriented gradients. Fixed-time jerk columns are exact; the free-time column uses a bounded one-sided difference with valid room.

hs3/+hs3Internal/+polynomial/convertPowerToBernstein.m

Converts ascending power coefficients to same-degree Bernstein coefficients on $[0, 1]$ through a cached exact Pascal-based matrix. Continuous-bound and corridor logic in both products use it.

hs3/+hs3Internal/+polynomial/createAffineSensitivityModel.m

Builds exact maps from scalar-coordinate jerk ordinates to segment polynomial coefficients, terminal P/V/A, and requested global-τ evaluations. These maps explain the fixed-time affine constraint structure.

hs3/+hs3Internal/+polynomial/createSubintervalBernsteinMap.m

Maps a normalized point/subinterval to its owning segment and creates an exact restricted power-to-Bernstein hull map. It validates that a constraint interval does not cross a segment boundary.

hs3/+hs3Internal/+polynomial/createTrajectoryPolynomial.m

Exactly integrates start/mid/end quadratic jerk across equal segments, propagating state continuity and returning quintic position, quartic velocity, cubic acceleration, quadratic jerk, segment times, and terminal state.

hs3/+hs3Internal/+polynomial/evaluateIntegratedSquaredJerk.m

Computes the exact integrated squared-jerk objective, decision gradient, and fixed-time Hessian. The free-time gradient includes the duration dependence; a constant Hessian is intentionally available only for fixed time.

hs3/+hs3Internal/+polynomial/evaluateTrajectoryPolynomial.m

Evaluates ascending-power segment records at absolute times, automatically selects the owning segment or accepts explicit indices, clamps local coordinates to $[0, 1]$, and returns N-by-D histories.

10.8 Benchmarks

benchmarks/benchmarkRandomMovingPolygonStress.m

Runs a deterministic stress sweep of large rotating/translating/deforming polygons. It preserves the exact request, planner result, independent validation, analytic boundary witness, seed, and per-case diagnostics rather than regenerating failed cases.

benchmarks/benchmarkStandaloneHs3Scaling.m

Measures repeated-turn and hairpin scaling serially. It builds shared scenarios, enforces HS3 result contracts, validates, and records route, solver, search, runtime, and environment evidence for each repeat and scale.

benchmarks/compareHs3ExtractionBaseline.m

Compares the current neutral HS3 extraction with a frozen historical baseline. It runs candidate reports, checks nested exact/numeric fields, reconstructs controls from polynomials, and compares serial runtime medians without rewriting the planner.

benchmarks/createRepeatedTurnBenchmarkScenario.m

Constructs one scale-controlled alternating-barrier request shared by scaling work. Turn count and options determine canonical obstacles, endpoints, limits, and documented constants; planner logic remains scenario-independent.

10.9 Maintained examples and helpers

Every scenario below follows the maintained pattern: resolve controls, create canonical obstacles and request data, call the public planner or intercept wrapper, validate independently, and plot the returned result when enabled. Several deterministic moving-scene inputs use the quintic smootherstep blend $10u^3 - 15u^4 + 6u^5$, associated with Perlin's improved-noise construction [5]. In this repository it is scenario input construction, not an HS3 motion model.

examples/exampleAlternatingSlalom.m

Demonstrates an input-driven route through alternating static barriers from $[-8, 0]$ to $[8, 0]$, exercising multiple turns without hidden waypoints.

examples/exampleDenseConcaveObstacle.m

Demonstrates protected dense concave geometry and visibility-based topology selection without scenario-specific planner guidance.

examples/exampleFortyMovingCircleGrid.m

Plans earliest diagonal motion through an 8-by-5 grid of 40 circles moving vertically, stressing timed visibility, moving associations, and independent validation.

examples/exampleFourAcceleratingCircles.m

Uses four vertically accelerating circles and a moving target to demonstrate interception through an early center-line opening.

examples/exampleInterceptMovingTargetAtSetTime.m

Runs an obstacle-free position-only intercept of a curved sampled target at a specified time and validates the wrapper's terminal target interpretation.

examples/exampleInterceptMovingTargetEarliest.m

Uses the public intercept wrapper to find the earliest observed feasible obstacle-free intercept within its bounded chronological search.

examples/exampleMovingBarrierWait.m

Demonstrates a trajectory that waits continuously for a translating barrier, exercising timed seeds and nonzero event timing rather than a spatial detour alone.

examples/exampleMovingCircleNoWrap.m

Demonstrates an immediate detour around a rising circle with periodic x explicitly disabled, protecting the non-wrapped obstacle contract.

examples/exampleMovingDeformingUSOutlineVisibility.m

Creates a dense U.S. outline that grows, deforms, rotates 180 coordinate units, and disappears, with a second moving sun obstacle. It exercises changing topology/visibility and the geographic helper.

examples/exampleNoPath.m

Constructs a full-height wall and expects a stable `noValidatedSeed` result. It independently validates the failure record and demonstrates workspace plus time-expanded diagnostics when plotting is enabled.

examples/exampleObstacleAvoidance.m

The compact deterministic one-rectangle example, demonstrating automatic side choice, protected geometry, HS3 motion, validation, and result-driven plotting.

examples/exampleObstacleFree.m

Exercises earliest-arrival rest-to-rest HS3 motion without geometry, isolating kinematic bounds, arrival optimization, and polynomial validation.

examples/exampleOpeningUShapedObstacle.m

Waits for a U-shaped obstacle opening and adds a scenario-local post-validation that the timed opening, rather than an unintended route, was used.

examples/exampleStaticUShapedObstacle.m

Demonstrates escape from a protected static U cavity through visibility/route class proposals without hidden waypoints.

examples/exampleStraightTargetAlternatingOcclusion.m

Tracks a straight moving target past square, circle, star, and U obstacles, checking blocked/unblocked intervals and a validated intercept in a free gap.

examples/exampleTargetExitsObstacle.m

The target begins inside a circle and later exits into free space while another obstacle affects transit; the wrapper must avoid declaring intercept while the target is occupied.

examples/exampleTwoOpposingUVisibilityGraph.m

Exercises automatic visibility and distinct route class choices around two opposing protected U obstacles.

examples/exampleUSOutlineExtremeVisibility.m

Runs sequential static Hawaii, Croatia, and Philippines dense-outline cases using bounded extreme visibility candidates and geographic scenario construction.

examples/private/createContiguousUSObstacle.m

Loads and unions a Mapping Toolbox U.S. outline, then creates static or extreme dynamic protected history. It is private to the deforming-U.S. example and keeps vertex-level geographic detail out of the top-level scenario flow.

examples/private/createGeographicRegionObstacle.m

Loads, clips, and densifies named geographic geometry, derives a directly blocked request without selecting a detour, and returns protected obstacle/history/scenario records for the extreme-outline example.

examples/resolveExampleOptions.m

The uniform example control router. It materializes public planner defaults, merges runtime/display overrides, normalizes text/logical aliases, and separates planner choices from plot/animation controls.

examples/validateExampleResult.m

A shared independent validator for example success/failure contracts. It checks planner validation, diagnostic consistency, optional direct-route expectations, and returns an example-local record without mutating the planner result.

10.10 Sandbox tools

sandbox/obstacleAvoidanceSandbox.m

A persistent MATLAB GUI with separate Goal and Free tabs. It turns drawn paths into canonical obstacles, supports segmented free planning, calls/validates/plots the public planner, maintains logs and controls, and exposes reset, undo, diagnostics, and export. Its UI policy is not production planner logic.

sandbox/exportSandboxDiagnosis.m

Writes a handle-free MAT diagnosis bundle for pre-run, completed, or failed sandbox state. The bundle retains the exact request, result, validation, scene, controls, logs, planning state, and reproduction guidance and verifies that the written file is nonempty.

10.11 Test support and suites

tests/+testSupport/plannerFixtures.m

Provides deterministic state, limit, rectangle obstacle, and representative polynomial trajectory builders so tests share physically identical fixtures and do not duplicate setup logic.

tests/+testSupport/verifySharedPlannerContract.m

Runs a reusable behavioral contract covering wrap policy, between-sample collision/dynamics, determinism, moving targets/obstacles, safety margin once, topology changes, stable diagnostics, and migration errors against the maintained planner.

tests/testArchitectureBoundaries.m

Enforces the two production roots, one public neutral HS3 entry, internal package topology, unique production basenames, neutral-source dependency direction, numerical solver ownership, and absence of removed legacy packages.

tests/testExampleContracts.m

Freezes selected large-example physical hashes and timing/arrival policies, verifies normal planner result schemas, checks default materialization, and ensures every maintained example routes the shared jerk override.

tests/testHs3OptionOwner.m

Tests public/default option equality, partial and empty override normalization, warn-once unknown fields that do not change behavior, and established error identifiers for invalid contracts.

tests/testHs3Planner.m

The main obstacle-aware integration suite. It covers public/direct calls, stable schemas, fixed/earliest behavior, direct-line preservation, topology/route class/timed seeds, clustering/envelopes, moving/deforming obstacles, continuous bounds/collision, margins, no-path, plotting, determinism, and moving-target wrappers.

tests/testHs3PolynomialOperations.m

Uses randomized coefficient parity and directional finite differences to verify affine sensitivities, fixed-time matrices, free-time/corridor gradients, subinterval restriction, and Bernstein continuous-bound calculations.

tests/testObstacleAvoidanceSandboxDiagnosis.m

Exercises diagnosis bundle round trips and reproduction for success, failure, and pre-run states, plus the persistent sandbox's public export actions and cleanup.

tests/testObstacleInfrastructure.m

Freezes canonical obstacle order/schema, normalizer equivalence, warnings, occupancy/query behavior, ordered boundary metadata, changing-history classification, and deterministic edge ordering.

tests/testPlannerStageTiming.m

Checks collision and validation timing ownership, shared success/failure timing schemas, solver work reconciliation, additive totals, and rejection of impossible over-attribution.

tests/testStandaloneHs3Kernel.m

Exercises the neutral kernel in one, two, and three dimensions; fixed/free time; asymmetric bounds; nonzero endpoint derivatives; expected infeasibility; affine point/interval rows; defaults; stable schemas; and dependency isolation.

11 Tracked non-MATLAB support artifacts

The current commit also contains 17 tracked non-MATLAB files:

- `README.md`: public architecture, API, examples, limitations, and requirements.
- `branch_assessment.md`: cumulative measured strengths, weaknesses, retained/rejected experiments, runtime, and coverage evidence.
- `verification.md`: historical commands, example matrices, comparisons, and verification limitations.
- `citation.md`: references for route class search, direct collocation, scenario motion, and Bernstein bounds.
- `plan.md`: HS3-only branch objective, completed cutover, startup verification limitation, and sandbox follow-up.
- `short_file_rationale.md`: rationale for retaining small public boundaries and focused HS3 helpers.
- `benchmark.csv`: chronological maintained-example measurements.
- `benchmarks/repeated_turn_hs3_phase_a.csv`: frozen repeated-turn scaling baseline.
- `AGENTS.md`: repository correctness, generality, interface, diagnostics, style, and verification rules.
- `.gitignore`: generated and scratch artifact exclusions.
- Seven files under `.agents/skills/`: local bounded-change, engineering-report, failure-diagnosis, and repository-health workflows plus their reference/metadata files. These govern agent work; they are not runtime planner inputs.

No tracked PDF, raster image, MAT file, or LaTeX source existed at the documented commit.

12 Traceability summary

Question	Primary implementation owners
What enters the public planner?	`planTrajectory.m`, `normalizePlannerRequest.m`, `resolveHs3Options.m`
Where is protection applied?	`createObstacle.m`; original and protected histories are retained separately.
How is dynamic geometry evaluated?	`prepareObstacles.m`, `shapeAtTime.m`, `queryObstacleOccupancyAtTime.m`
How is route topology proposed?	`createRouteCandidates.m`, `timeExpandedVisibilitySearch.m`, `denseSweptEnvelope.m`
How does a polygon become an HS3 constraint?	`solveRouteCandidate.m`, `createSeedCorridor.m`, `evaluateTrajectoryConstraints.m`, `createConstraintMatrices.m`
Where is HS3 reconstructed?	`createTrajectoryPolynomial.m`, `evaluateTrajectoryPolynomial.m`
Where are continuous bounds certified?	`convertPowerToBernstein.m`, `createSubintervalBernsteinMap.m`, planner/neutral constraint evaluators
Which numerical solver is used?	`hs3Internal/+solver/optimize.m`: `quadprog` fixed time, `fmincon` free time/recovery.

Question	Primary implementation owners
What finally authorizes success?	`validateTrajectory.m` for the X/Y planner; `hs3Internal/validate.m` for the public neutral engine.

13 References

References

[1] HS3 repository source tree. *HS3-planner branch at commit* `ce0dfdd42c5cf186761603fa18d14c5ba4583551`. MATLAB implementation, tests, examples, benchmarks, and repository records as cited by file and line number throughout this report. Snapshot inspected 2026-08-27.

[2] S. Moreno-Martin, L. Ros, and E. Celaya. "Collocation Methods for Second and Higher Order Systems." *Autonomous Robots*, 48, Article 2, 2024. doi:10.1007/s10514-023-10155-z.

[3] R. T. Farouki. "The Bernstein Polynomial Basis: A Centennial Retrospective." *Computer Aided Geometric Design*, 29(6):379–419, 2012. doi:10.1016/j.cagd.2012.03.001.

[4] S. Bhattacharya, M. Likhachev, and V. Kumar. "Search-Based Path Planning with Homotopy Class Constraints in 3D." *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, pages 2097–2099, 2012. doi:10.1609/aaai.v26i1.8435.

[5] K. Perlin. "Improving Noise." *Proceedings of SIGGRAPH 2002*, pages 681–682, 2002. doi:10.1145/566570.566636.

[6] MathWorks. "`quadprog` - Quadratic Programming." MATLAB Optimization Toolbox documentation. `https://www.mathworks.com/help/optim/ug/quadprog.html`. Accessed 2026-08-27.

[7] MathWorks. "`fmincon` - Constrained Nonlinear Multivariable Minimization." MATLAB Optimization Toolbox documentation. `https://www.mathworks.com/help/optim/ug/fmincon.html`. Accessed 2026-08-27.

[8] MathWorks. "`polybuffer` - Create Buffer Around Points, Lines, or `polyshape` Objects." MATLAB documentation. `https://www.mathworks.com/help/matlab/ref/polyshape.polybuffer.html`. Accessed 2026-08-27.

Closing statement

At this commit, HS3 is best understood as a reusable exact polynomial and optimization kernel embedded inside a larger, explicitly bounded planning system. Polygon histories first influence topology proposals, then become local exterior half-spaces for the jerk-control optimization. They never become hidden waypoints, and they are never reduced to a one-time sample for final acceptance. The complete protected polygon history, continuous polynomial bounds, and independent validator remain the final authority.